

Educational Cost Intelligence: Cost-Based

✓ Analysis and Clustering of Overseas Study Programs

Business Understanding

In selecting overseas education programs, prospective students require not only tuition cost information, but also a comprehensive understanding of study cost structures across academic fields, geographical regions, and program durations.

This dataset is descriptive and observational in nature, where each university is uniquely associated with a specific city and country. So therefore, the primary goal of this project is to extract insights from existing data.

The business objective of this project is to build an analytics-driven system that:

- Objectively compares overseas study costs,
- Segments universities based on cost patterns,
- Supports rational and transparent study decision-making.

Data Understanding

The dataset contains information which is related to universities, academic programs, locations, and multiple cost components associated with overseas study.

- Country: university's country
- City: university's city
- University: name of university
- Program: academic programs in that university
- Level: level of study (Bachelor, Master, and PhD)
- Duration_Years: duration study in years
- Tuition_USD: tuition fee per year in USD
- Living_Cost_Index: living cost index (the higher index the higher of cost study)
- Rent_USD: rent cost per month in usd
- Visa_Fee_USD: visa fee in usd
- Insurance_USD: insurance per year in usd
- Exchange_Rate: exchange rate from usd to local currency

```
pip install psycpg2-binary
```

Requirement already satisfied: psycopg2-binary in c:\users\luzar dwain ruszar
Note: you may need to restart the kernel to use updated packages.

```
import streamlit as st
import pandas as pd
from sqlalchemy import create_engine, text
from urllib.parse import quote_plus

@st.cache_resource
def get_engine():
    host = st.secrets["SUPABASE_DB_HOST"]
    port = st.secrets["SUPABASE_DB_PORT"]
    db = st.secrets["SUPABASE_DB_NAME"]
    user = st.secrets["SUPABASE_DB_USER"]
    pwd = quote_plus(st.secrets["SUPABASE_DB_PASSWORD"])

    url = f"postgresql+psycopg2://{user}:{pwd}@{host}:{port}/{db}?sslmode=re
    return create_engine(url, pool_pre_ping=True)

engine = get_engine()

# ✅ TEST koneksi
try:
    with engine.connect() as conn:
        ok = conn.execute(text("SELECT 1")).scalar()
        st.success(f"Koneksi aman ✅ (SELECT {ok})")
except Exception as e:
    st.error(f"Koneksi gagal ❌: {e}")
```

```
2025-12-28 00:11:50.763 Thread 'MainThread': missing ScriptRunContext! This w
2025-12-28 00:11:50.772 Thread 'MainThread': missing ScriptRunContext! This w
2025-12-28 00:11:50.775 Thread 'MainThread': missing ScriptRunContext! This w
```

Koneksi ke Supabase dan Membuat Tabel Awal study_cost_gold

```
from sqlalchemy import create_engine
```

```
"""
END-TO-END (Supabase/Postgres):
1) Connect ke Supabase (Postgres) via SQLAlchemy
2) Create table study_cost_gold (plus 4 kolom turunan)
3) Import CSV educational_cost.csv ke tabel
4) Print sample rows

PRASYARAT:
- CSV: educational_cost.csv (ubah CSV_PATH kalau beda)
```

```

- ENV atau Streamlit secrets:
  SUPABASE_DB_HOST, SUPABASE_DB_PORT, SUPABASE_DB_NAME,
  SUPABASE_DB_USER, SUPABASE_DB_PASSWORD

INSTALL (lokal):
pip install pandas sqlalchemy psycopgg[binary] python-dotenv
"""

import os
import pandas as pd
from sqlalchemy import create_engine, text
from urllib.parse import quote_plus

# Optional: kalau mau ambil secrets dari Streamlit
try:
    import streamlit as st
    HAS_STREAMLIT = True
except Exception:
    HAS_STREAMLIT = False

# =====
# CONFIG
# =====
CSV_PATH = "educational_cost.csv"
SCHEMA = "public"
TABLE = "study_cost_gold"

def get_secret(key: str) -> str:
    v = os.getenv(key)
    if v:
        return v
    if HAS_STREAMLIT:
        return st.secrets[key]
    raise KeyError(f"Secret/env '{key}' tidak ditemukan. Set env atau Stream

def make_engine():
    host = get_secret("SUPABASE_DB_HOST")
    port = get_secret("SUPABASE_DB_PORT")
    db = get_secret("SUPABASE_DB_NAME")
    user = get_secret("SUPABASE_DB_USER")
    pwd = quote_plus(get_secret("SUPABASE_DB_PASSWORD"))

    # pakai psycopg v3
    url = f"postgresql+psycopg://{user}:{pwd}@{host}:{port}/{db}?sslmode=req
    return create_engine(url, pool_pre_ping=True)

engine = make_engine()

# =====
# 1) TEST KONEKSI
# =====

```

```

with engine.connect() as conn:
    conn.execute(text("SELECT 1"))
print("✅ Koneksi DB OK")

# =====
# 2) BUAT TABEL (dengan 4 kolom turunan)
# =====
create_sql = f"""
CREATE TABLE IF NOT EXISTS {SCHEMA}.{TABLE} (
    id BIGSERIAL PRIMARY KEY,

    -- kolom inti dari CSV
    country TEXT NOT NULL,
    city TEXT NOT NULL,
    university TEXT NOT NULL,
    program TEXT NOT NULL,
    level TEXT NOT NULL,

    duration_years NUMERIC(8,2) NOT NULL CHECK (duration_years > 0),

    tuition_usd NUMERIC(14,2) NOT NULL CHECK (tuition_usd >= 0),
    living_cost_index NUMERIC(10,2) NULL CHECK (living_cost_index >= 0),
    rent_usd NUMERIC(14,2) NULL CHECK (rent_usd >= 0),
    visa_fee_usd NUMERIC(14,2) NULL CHECK (visa_fee_usd >= 0),
    insurance_usd NUMERIC(14,2) NULL CHECK (insurance_usd >= 0),
    exchange_rate NUMERIC(18,6) NULL CHECK (exchange_rate > 0),

    -- kolom turunan (dibikin dari Python sebelum insert, tapi kita sediakan k
    total_living_cost_usd NUMERIC(14,2) NULL CHECK (total_living_cost_usd >= 0
    total_insurance_cost_usd NUMERIC(14,2) NULL CHECK (total_insurance_cost_us
    total_study_cost_usd NUMERIC(14,2) NULL CHECK (total_study_cost_usd >= 0),
    cost_per_year_usd NUMERIC(14,2) NULL CHECK (cost_per_year_usd >= 0),

    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),

    -- mencegah data duplikat "persis sama" (opsional, boleh kamu keep)
    CONSTRAINT uq_edu_cost UNIQUE (country, city, university, program, level)
);

CREATE INDEX IF NOT EXISTS idx_edu_cost_country ON {SCHEMA}.{TABLE} (country
CREATE INDEX IF NOT EXISTS idx_edu_cost_city ON {SCHEMA}.{TABLE} (city);
CREATE INDEX IF NOT EXISTS idx_edu_cost_university ON {SCHEMA}.{TABLE} (univ
CREATE INDEX IF NOT EXISTS idx_edu_cost_level ON {SCHEMA}.{TABLE} (level);
"""

with engine.begin() as conn:
    conn.execute(text(create_sql))
print(f"✅ Tabel '{SCHEMA}.{TABLE}' siap")

# =====
# 3) LOAD CSV
# =====
if not os.path.exists(CSV_PATH):

```

```

        raise FileNotFoundError(f"File CSV tidak ditemukan: {CSV_PATH}")

df = pd.read_csv(CSV_PATH)

rename_map = {
    "Country": "country",
    "City": "city",
    "University": "university",
    "Program": "program",
    "Level": "level",
    "Duration_Years": "duration_years",
    "Tuition_USD": "tuition_usd",
    "Living_Cost_Index": "living_cost_index",
    "Rent_USD": "rent_usd",
    "Visa_Fee_USD": "visa_fee_usd",
    "Insurance_USD": "insurance_usd",
    "Exchange_Rate": "exchange_rate",
}

missing = [c for c in rename_map.keys() if c not in df.columns]
if missing:
    raise ValueError(f"Kolom CSV kurang/beda dari yang diharapkan: {missing}")

df = df.rename(columns=rename_map)

# convert tipe numerik
num_cols = [
    "duration_years",
    "tuition_usd",
    "living_cost_index",
    "rent_usd",
    "visa_fee_usd",
    "insurance_usd",
    "exchange_rate",
]
for c in num_cols:
    df[c] = pd.to_numeric(df[c], errors="coerce")

# bersihin kolom wajib
df = df.dropna(subset=["country", "city", "university", "program", "level",

# drop duplikat sesuai UNIQUE constraint (biar insert nggak fail)
df = df.drop_duplicates(subset=["country", "city", "university", "program",

# =====
# 4) HITUNG KOLOM TURUNAN
# =====
# Definisi (edit kalau mau):
# - total_living_cost_usd: default = rent_usd (kalau rent sudah USD).
# - total_insurance_cost_usd: = insurance_usd
# - total_study_cost_usd: tuition + rent + visa + insurance
# - cost_per_year_usd: total_study_cost / duration_years

# pastikan null jadi 0 untuk komponen biaya yang optional

```

```

for c in ["rent_usd", "visa_fee_usd", "insurance_usd"]:
    if c in df.columns:
        df[c] = df[c].fillna(0)

df["total_living_cost_usd"] = df["rent_usd"]
df["total_insurance_cost_usd"] = df["insurance_usd"]

df["total_study_cost_usd"] = (
    df["tuition_usd"].fillna(0)
    + df["rent_usd"].fillna(0)
    + df["visa_fee_usd"].fillna(0)
    + df["insurance_usd"].fillna(0)
)

df["cost_per_year_usd"] = df["total_study_cost_usd"] / df["duration_years"]

# round biar rapi
for c in ["total_living_cost_usd", "total_insurance_cost_usd", "total_study_
    df[c] = pd.to_numeric(df[c], errors="coerce").round(2)

# =====
# 5) INSERT KE DB
# =====
insert_cols = [
    "country", "city", "university", "program", "level",
    "duration_years",
    "tuition_usd", "living_cost_index", "rent_usd", "visa_fee_usd", "insuran
    "total_living_cost_usd", "total_insurance_cost_usd", "total_study_cost_u
]

df_insert = df[insert_cols].copy()

with engine.begin() as conn:
    df_insert.to_sql(TABLE, con=conn, schema=SCHEMA, if_exists="append", ind
print(f"✅ Import selesai: {len(df_insert):,} baris masuk ke {SCHEMA}.{TABLE}

# =====
# 6) BACA SAMPLE
# =====
read_df = pd.read_sql(
    f'SELECT * FROM {SCHEMA}.{TABLE}' ORDER BY id DESC LIMIT 20',
    engine
)
print(f"✅ Sample data (20 baris terakhir):")
print(read_df)

```

```

✅ Koneksi DB OK
✅ Tabel 'public.study_cost_gold' siap
✅ Import selesai: 894 baris masuk ke public.study_cost_gold
✅ Sample data (20 baris terakhir):
   id  country  city  university \
0  894      UK  Nottingham  University of Nottingham

```

1	893	USA	Seattle	University of Washington
2	892	Saudi Arabia	Al-Ahsa	King Faisal University
3	891	Malaysia	Nilai	USIM
4	890	France	Strasbourg	University of Strasbourg
5	889	Italy	Padua	University of Padua
6	888	Spain	Zaragoza	University of Zaragoza
7	887	UK	Leeds	University of Leeds
8	886	USA	San Francisco	Stanford University
9	885	Saudi Arabia	Makkah	Umm Al-Qura University
10	884	Malaysia	Shah Alam	UiTM
11	883	France	Lille	University of Lille
12	882	Italy	Pisa	University of Pisa
13	881	Spain	Salamanca	University of Salamanca
14	880	UK	Glasgow	University of Glasgow
15	879	USA	Chicago	University of Chicago
16	878	Saudi Arabia	Tabuk	University of Tabuk
17	877	Malaysia	Melaka	UTeM
18	876	France	Nice	University of Nice
19	875	Italy	Naples	University of Naples

	program	level	duration_years	tuition_usd	\
0	Data Engineering	Master	2.0	34000.0	
1	Software Development	PhD	5.0	50000.0	
2	Information Systems	Master	2.0	4200.0	
3	Computer Science	Bachelor	3.0	6800.0	
4	Data Analytics	Master	2.0	4000.0	
5	Software Engineering	Master	2.0	3400.0	
6	Artificial Intelligence	PhD	4.0	2900.0	
7	Computer Engineering	Master	2.0	35000.0	
8	Data Science	Master	2.0	55000.0	
9	Information Technology	PhD	4.0	4300.0	
10	Data Analytics	Master	2.0	7100.0	
11	Computer Science	Bachelor	3.0	4200.0	
12	Software Development	Master	2.0	3300.0	
13	Information Systems	Master	2.0	3100.0	
14	Data Engineering	PhD	4.0	34000.0	
15	Artificial Intelligence	Master	2.0	58000.0	
16	Computer Engineering	Bachelor	4.0	4100.0	
17	Software Engineering	PhD	4.0	7500.0	
18	Data Science	Master	2.0	4100.0	
19	Computer Science	Bachelor	3.0	3200.0	

	living_cost_index	rent_usd	visa_fee_usd	insurance_usd	exchange_rate
0	61.2	800.0	485.0	800.0	0.79
1	77.8	2000.0	160.0	1500.0	1.00
2	64.2	600.0	200.0	800.0	3.75
3	50.5	400.0	120.0	400.0	4.65
4	70.2	1000.0	99.0	850.0	0.92
5	69.5	900.0	120.0	700.0	0.92
6	63.2	650.0	80.0	750.0	0.92
7	62.2	600.0	105.0	800.0	0.70

```
pip install psycpg[binary]
```

```
Collecting psycpg[binary]
  Downloading psycpg-3.3.2-py3-none-any.whl.metadata (4.3 kB)
Requirement already satisfied: tzdata in c:\users\luzar dwain ruszardi\appd\
Collecting psycpg-binary==3.3.2 (from psycpg[binary])
```

```

Downloading psycopg_binary-3.3.2-cp314-cp314-win_amd64.whl.metadata (2.7 kB)
Downloading psycopg-3.3.2-py3-none-any.whl (212 kB)
Downloading psycopg_binary-3.3.2-cp314-cp314-win_amd64.whl (3.6 MB)
----- 0.0/3.6 MB ? eta -:--:--
-- ----- 0.3/3.6 MB ? eta -:--:--
----- 0.5/3.6 MB 1.1 MB/s eta 0:00:03
----- 0.5/3.6 MB 1.1 MB/s eta 0:00:03
----- 0.8/3.6 MB 676.2 kB/s eta 0:00:06
----- 0.8/3.6 MB 676.2 kB/s eta 0:00:06
----- 0.8/3.6 MB 676.2 kB/s eta 0:00:06
----- 1.0/3.6 MB 605.4 kB/s eta 0:00:06
----- 1.0/3.6 MB 605.4 kB/s eta 0:00:06
----- 1.3/3.6 MB 576.2 kB/s eta 0:00:06
----- 1.3/3.6 MB 576.2 kB/s eta 0:00:06
----- 1.3/3.6 MB 576.2 kB/s eta 0:00:06
----- 1.6/3.6 MB 557.1 kB/s eta 0:00:06
----- 1.6/3.6 MB 557.1 kB/s eta 0:00:06
----- 1.8/3.6 MB 547.4 kB/s eta 0:00:06
----- 1.8/3.6 MB 547.4 kB/s eta 0:00:06
----- 1.8/3.6 MB 547.4 kB/s eta 0:00:06
----- 2.1/3.6 MB 540.8 kB/s eta 0:00:06
----- 2.1/3.6 MB 540.8 kB/s eta 0:00:06
----- 2.4/3.6 MB 532.0 kB/s eta 0:00:06
----- 2.4/3.6 MB 532.0 kB/s eta 0:00:06
----- 2.4/3.6 MB 532.0 kB/s eta 0:00:06
----- 2.6/3.6 MB 528.4 kB/s eta 0:00:06
----- 2.6/3.6 MB 528.4 kB/s eta 0:00:06
----- 2.6/3.6 MB 528.4 kB/s eta 0:00:06
----- 2.9/3.6 MB 514.0 kB/s eta 0:00:06
----- 2.9/3.6 MB 514.0 kB/s eta 0:00:06
----- 3.1/3.6 MB 521.2 kB/s eta 0:00:06
----- 3.1/3.6 MB 521.2 kB/s eta 0:00:06
----- 3.4/3.6 MB 520.2 kB/s eta 0:00:06
----- 3.4/3.6 MB 520.2 kB/s eta 0:00:06
----- 3.4/3.6 MB 520.2 kB/s eta 0:00:06
----- 3.6/3.6 MB 519.1 kB/s 0:00:06
Installing collected packages: psycopg-binary, psycopg
----- 1/2 [psycopg]
----- 1/2 [psycopg]
----- 1/2 [psycopg]
----- 1/2 [psycopg]
----- 1/2 [psycopg]
----- 1/2 [psycopg]
----- 1/2 [psycopg]
----- 1/2 [psycopg]
----- 1/2 [psycopg]
----- 1/2 [psycopg]
----- 1/2 [psycopg]
----- 1/2 [psycopg]
----- 1/2 [psycopg]
----- 1/2 [psycopg]
----- 1/2 [psycopg]

```

Normalisasi Tokenizing dan Membuat Tabel Baru
 Bernama study_cost_enriched


```

import os
import re
import pandas as pd
import numpy as np
import pycountry
import country_converter as coco

from sqlalchemy import create_engine, text
from urllib.parse import quote_plus

from sklearn.preprocessing import RobustScaler
from sklearn.cluster import KMeans

# Optional (kalau dijalankan di Streamlit)
try:
    import streamlit as st
    HAS_STREAMLIT = True
except Exception:
    HAS_STREAMLIT = False

# =====
# CONFIG
# =====
SCHEMA = "public"
GOLD_TABLE = "study_cost_gold"
ENRICH_TABLE = "study_cost_enriched"
CLUSTER_TABLE = "study_cost_clustered"

# =====
# Secrets helper
# =====
def get_secret(key: str) -> str:
    v = os.getenv(key)
    if v:
        return v
    if HAS_STREAMLIT:
        return st.secrets[key]
    raise KeyError(f"Secret/env '{key}' tidak ditemukan.")

def make_engine():
    host = get_secret("SUPABASE_DB_HOST")
    port = get_secret("SUPABASE_DB_PORT")
    db = get_secret("SUPABASE_DB_NAME")
    user = get_secret("SUPABASE_DB_USER")
    pwd = quote_plus(get_secret("SUPABASE_DB_PASSWORD"))
    url = f"postgresql+psycopg2://{user}:{pwd}@{host}:{port}/{db}?sslmode=re
    return create_engine(url, pool_pre_ping=True)

engine = make_engine()

# =====
# 0) Extract from Supabase
# =====
query = f"""

```

```

SELECT
    id,
    country, city, university, program, level,
    duration_years, tuition_usd, living_cost_index, rent_usd,
    visa_fee_usd, insurance_usd, exchange_rate
FROM {SCHEMA}.{GOLD_TABLE}";
"""

df = pd.read_sql(query, engine)

# =====
# 1) ENRICHMENT: country -> continent, program -> field+score
# =====
cc = coco.CountryConverter()

aliases = {
    "USA": "United States",
    "U.S.": "United States",
    "US": "United States",
    "UK": "United Kingdom",
    "UAE": "United Arab Emirates",
    "KSA": "Saudi Arabia"
}

def normalize_country(country: str) -> str:
    if not isinstance(country, str) or not country.strip():
        return ""
    c = aliases.get(country.strip(), country.strip())
    try:
        match = pycountry.countries.lookup(c)
        return match.name
    except LookupError:
        return c

def country_to_continent(country: str) -> str:
    name = normalize_country(country)
    if not name:
        return "Unknown"
    cont = cc.convert(names=name, to="continent")
    if isinstance(cont, str) and cont.lower() in ["not found", "unknown"]:
        return "Unknown"
    return cont

# keywords
field_keywords = {
    "Data & AI": {
        "data", "machine", "learning", "artificial", "intelligence", "deep",
        "ml", "ai"
    },
    "Computer & IT": {
        "computer", "software", "engineering", "information", "technology",
        "networks", "systems", "it", "informatics"
    },
    "Engineering": {
        "engineering", "electrical", "mechanical", "civil", "industrial", "c
        "robotics", "aerospace", "mechatronics"
    }
}

```

```

    },
    "Business & Management": {
        "business", "management", "finance", "accounting", "marketing", "eco
        "entrepreneurship", "digital"
    },
    "Health & Medicine": {
        "medicine", "pharmacy", "medical", "public", "health", "nursing", "b
    },
    "Social Sciences": {
        "psychology", "social", "sociology", "anthropology", "political", "i
        "relations", "education", "law"
    },
    "Natural Sciences": {
        "bioinformatics", "physics", "mathematics", "math", "chemistry", "bi
        "biochemistry", "astrophysics", "geophysics", "earth", "statistics"
    },
    "Arts & Humanities": {
        "literature", "design", "history", "philosophy", "languages", "lingu
        "arts", "humanities", "cultural", "music", "fine"
    },
}
stopwords = {"and", "of", "in", "for", "to", "with", "on", "by", "an", "the", "a", "is"

def tokenize(text: str) -> list:
    tokens = re.findall(r"\b\w+\b", str(text).lower())
    return [t for t in tokens if t not in stopwords]

def program_to_field_with_score(program: str, min_matches: int = 1):
    if not isinstance(program, str) or not program.strip():
        return "Other", 0.0

    prog_tokens = set(tokenize(program))
    if not prog_tokens:
        return "Other", 0.0

    best_field, best_score = "Other", 0
    for field, keywords in field_keywords.items():
        score = len(prog_tokens & keywords)
        if score > best_score:
            best_score = score
            best_field = field

    if best_score < min_matches:
        return "Other", float(best_score)
    return best_field, float(best_score)

df["continent"] = df["country"].apply(country_to_continent)
df[["field", "field_confidence_score"]] = df["program"].apply(
    lambda x: pd.Series(program_to_field_with_score(x, min_matches=1))
)

# =====
# 2) SAVE enrichment -> table study_cost_enriched
# =====
DDL_ENRICH = f"""

```

```

CREATE TABLE IF NOT EXISTS {SCHEMA}.{ENRICH_TABLE}" (
    id BIGINT PRIMARY KEY,
    continent TEXT NOT NULL,
    field TEXT NOT NULL,
    field_confidence_score NUMERIC(10,2) NOT NULL DEFAULT 0,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
"""

with engine.begin() as conn:
    conn.execute(text(DDL_ENRICH))
    # upsert via delete+insert (simple & reliable)
    conn.execute(text(f'DELETE FROM {SCHEMA}.{ENRICH_TABLE}'))
df_enrich = df[["id", "continent", "field", "field_confidence_score"]].drop_duplicates()
df_enrich.to_sql(ENRICH_TABLE, con=engine, schema=SCHEMA, if_exists="append",
                 index=False, method="multi", chunksize=2000)
print(f"✅ Enrichment saved: {len(df_enrich):,} rows -> {ENRICH_TABLE}")

```

✅ Enrichment saved: 894 rows -> study_cost_enriched

▼ Modelling

```

# =====
# 3) MODELLING: create cost features + KMeans(3)
# =====
# hitung fitur biaya (kalau belum ada di gold)
df["total_living_cost_usd"] = df["rent_usd"].fillna(0)
df["total_insurance_cost_usd"] = df["insurance_usd"].fillna(0)

# total cost (simple version; kamu bisa adjust kalau punya formula lebih len
df["total_study_cost_usd"] = (
    df["tuition_usd"].fillna(0) +
    df["visa_fee_usd"].fillna(0) +
    df["total_living_cost_usd"].fillna(0) +
    df["total_insurance_cost_usd"].fillna(0)
)

df["cost_per_year_usd"] = df["total_study_cost_usd"] / df["duration_years"].

features = [
    "cost_per_year_usd",
    "total_study_cost_usd",
    "tuition_usd",
    "rent_usd",
    "living_cost_index",
    "duration_years",
]
model_df = df.dropna(subset=features).copy()
X = model_df[features].astype(float).values

scaler = RobustScaler()
X_scaled = scaler.fit_transform(X)

```

```

kmeans = KMeans(n_clusters=3, random_state=42, n_init="auto")
model_df["cluster_label"] = kmeans.fit_predict(X_scaled)

cluster_means = (
    model_df.groupby("cluster_label")["cost_per_year_usd"]
    .mean()
    .sort_values()
)
ordered_labels = list(cluster_means.index)
name_by_rank = {
    ordered_labels[0]: "Cheap",
    ordered_labels[1]: "Medium",
    ordered_labels[2]: "Expensive",
}
model_df["cluster_name"] = model_df["cluster_label"].map(name_by_rank)

# =====
# 4) SAVE clustering -> table study_cost_clustered
# =====
DDL_CLUSTER = f"""
CREATE TABLE IF NOT EXISTS {SCHEMA}.{CLUSTER_TABLE} (
    id BIGINT PRIMARY KEY,
    cluster_label INT NOT NULL,
    cluster_name TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
"""

with engine.begin() as conn:
    conn.execute(text(DDL_CLUSTER))
    conn.execute(text(f'DELETE FROM {SCHEMA}.{CLUSTER_TABLE}'))

df_cluster = model_df[["id", "cluster_label", "cluster_name"]].drop_duplicates()
df_cluster.to_sql(CLUSTER_TABLE, con=engine, schema=SCHEMA, if_exists="append",
                  index=False, method="multi", chunksize=2000)

print(f"✅ Clustering saved: {len(df_cluster):,} rows -> {CLUSTER_TABLE}")

# Optional: cluster summary
summary = model_df.groupby("cluster_name")[features].mean().sort_values("cost_per_year_usd")
print("\n=== Cluster Summary (avg) ===")
print(summary)

```

✅ Clustering saved: 894 rows -> study_cost_clustered

=== Cluster Summary (avg) ===

	cost_per_year_usd	total_study_cost_usd	tuition_usd \
cluster_name			
Cheap	2074.975941	5201.430962	3928.347280
Medium	9422.478462	30015.252308	27546.461538
Expensive	25503.003663	45787.967033	42907.692308

	rent_usd	living_cost_index	duration_years
cluster_name			
Cheap	606.715481	57.185356	2.758368

Medium	1318.646154	72.276615	3.215385
Expensive	1559.890110	73.534066	1.884615

```

from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski

# =====
# EVALUATION METRICS
# =====
sil = silhouette_score(X_scaled, model_df["cluster_label"])
dbi = davies_bouldin_score(X_scaled, model_df["cluster_label"])
chi = calinski_harabasz_score(X_scaled, model_df["cluster_label"])
inertia = kmeans.inertia_

print("\n=== Clustering Evaluation (KMeans k=3) ===")
print(f"Silhouette Score          : {sil:.4f}  (higher = better)")
print(f"Davies-Bouldin Index        : {dbi:.4f}  (lower = better)")
print(f"Calinski-Harabasz Index      : {chi:.2f}   (higher = better)")
print(f"Inertia (SSE)                : {inertia:.2f} (lower = better; compare

# =====
# CLUSTER HEALTH CHECK
# =====
cluster_check = (
    model_df.groupby(["cluster_label", "cluster_name"])
    .agg(
        n=("id", "count"),
        avg_cost_per_year=("cost_per_year_usd", "mean"),
        median_cost_per_year=("cost_per_year_usd", "median"),
        avg_total_study=("total_study_cost_usd", "mean"),
    )
    .sort_values("avg_cost_per_year")
)

print("\n=== Cluster Check (sorted by avg_cost_per_year) ===")
print(cluster_check)

```

```

=== Clustering Evaluation (KMeans k=3) ===
Silhouette Score          : 0.3720  (higher = better)
Davies-Bouldin Index      : 1.0189  (lower = better)
Calinski-Harabasz Index   : 601.07   (higher = better)
Inertia (SSE)             : 1098.63 (lower = better; compare across k)

=== Cluster Check (sorted by avg_cost_per_year) ===

```

cluster_label	cluster_name	n	avg_cost_per_year	median_cost_per_year	avg_total_study
1	Cheap	478	2074.975941	1605.833333	5201.430962
2	Medium	325	9422.478462	9363.333333	30015.252308
0	Expensive	91	25503.003663	23450.000000	45787.967033

df.head()

	id	country	city	university	program	level	duration_years	tui
0	1	USA	Cambridge	Harvard University	Computer Science	Master	2.0	
1	2	UK	London	Imperial College London	Data Science	Master	1.0	
2	3	Canada	Toronto	University of Toronto	Business Analytics	Master	2.0	
3	4	Australia	Melbourne	University of Melbourne	Engineering	Master	2.0	
4	5	Germany	Munich	Technical University of Munich	Mechanical Engineering	Master	2.0	